

МІНІСТЕРСТВО ОСВІТИ ТА НАУКИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ЛЬВІВСЬКА ПОЛІТЕХНІКА



АВТОМАТИЗОВАНЕ
ПРОЕКТУВАННЯ КОМП'ЮТЕРНИХ
СИСТЕМ

Лабораторна робота №4
“Створення документації doxygen”

Виконав:
студент гр. КІ-404
Піскурський
С. В.

Прийняв:
Федак П. Р.

Львів – 2024

Завдання:

1. Add doxygen comments for all public functions, classes, properties, fields...
2. Generate documentation based on doxygen comments
3. Required steps

Теоретичний матеріал

Doxygen — це інструмент для автоматичної генерації документації з коментарів у вихідному коді. Він широко використовується для мов програмування C, C++, Java, Python, та багатьох інших. Коментарі, які відповідають певному синтаксису, використовуються для створення HTML, LaTeX, або інших форматів документації.

Основні можливості Doxygen:

1. **Автоматичне створення документації:** витягує інформацію з коментарів, описуючи функції, класи, файли, структури, змінні тощо.
2. **Підтримка різних форматів:** HTML, PDF, RTF, XML та інші.
3. **Генерація діаграм:** діаграми класів, викликів функцій тощо за допомогою Graphviz.
4. **Підтримка конфігурації:** файл Doxyfile задає налаштування для генерації документації.
5. **Перехресні посилання:** забезпечує зручну навігацію між елементами коду.

Типові теги Doxygen:

- @brief або \brief — короткий опис.
- @param або \param — опис параметрів функції.
- @return або \return — опис значення, що повертається функцією.
- @file — опис файлу.
- @author — автор коду.
- @see — посилання на інші частини документації.
- @warning — попередження.
- @note — примітка.

Код програми на C++ із коментарями Doxygen

```
/// @file main.cpp
/// @brief Демонстрація використання Doxygen.
/// @author Ваше ім'я

#include <iostream>

/// @brief Функція для обчислення суми двох чисел.
/// @param a Перше число.
/// @param b Друге число.
/// @return Сума чисел.
int add(int a, int b) {
    return a + b;
}
```

```

}

/// @brief Основна функція програми.
/// @return Код завершення програми.
int main() {
    int x = 5, y = 10;
    std::cout << "Сума: " << add(x, y) << std::endl;
    return 0;
}

```

Виконання:

Опис всіх дій, які були зроблені для додавання підтримки Doxygen у проект

1. Встановлення Doxygen

- Doxygen встановлений на локальному середовищі через пакетний менеджер:

```

sudo apt-get update
sudo apt-get install -y doxygen

```

- У CI/CD YAML-файл додано команду для встановлення Doxygen:

```

- name: Install Doxygen
  run: sudo apt-get install -y doxygen

```

2. Створення файлу Doxyfile.in

- У кореневій директорії проекту створено файл Doxyfile.in з налаштуваннями для Doxygen. Ось основний вміст:

```

PROJECT_NAME           = "Game Server"
OUTPUT_DIRECTORY       = "@CMAKE_CURRENT_SOURCE_DIR@/docs"
EXTRACT_ALL            = YES
EXTRACT_STATIC         = YES
INPUT                  = "@CMAKE_SOURCE_DIR@/include"
RECURSIVE              = YES
GENERATE_HTML          = YES
GENERATE_LATEX         = NO

```

- Використано @CMAKE_CURRENT_SOURCE_DIR@ для генерування документації у директорії build/docs.

3. Додавання коментарів у код

- Додані Doxygen-коментарі до всіх функцій, структур, перерахувань та глобальних змінних. Приклад:

```

/**
 * @brief Parses the configuration file and populates the configuration map.
 *
 * @param filename The name of the configuration file.
 * @param configMap The map to populate with configuration data.
 * @return true if the file was successfully parsed, false otherwise.
 */
bool parseConfigFile(const std::string &filename, std::unordered_map<std::string, std::string>
&configMap);

/**
 * @enum FINISHED

```

```

* @brief Represents the game completion status.
*/
enum class FINISHED {
    NOT_YET = -1, /**< Game is not finished. */
    DRAW,        /**< Game is a draw. */
    WINNER       /**< Game has a winner. */
};

```

4. Підтримка в CMake

- Додано підтримку Doxygen у CMakeLists.txt:

```

find_package(Doxygen REQUIRED)

if (DOXYGEN_FOUND)
    set(DOXYGEN_IN ${CMAKE_CURRENT_SOURCE_DIR}/Doxyfile.in)
    set(DOXYGEN_OUT ${CMAKE_CURRENT_BINARY_DIR}/Doxyfile)

    configure_file(${DOXYGEN_IN} ${DOXYGEN_OUT} @ONLY)

    add_custom_target(doc
        COMMAND ${DOXYGEN_EXECUTABLE} ${DOXYGEN_OUT}
        WORKING_DIRECTORY ${CMAKE_CURRENT_BINARY_DIR}
        COMMENT "Generating API documentation with Doxygen"
        VERBATIM
    )
else ()
    message("Doxygen needs to be installed to generate the documentation")
endif ()

```

5. Зміни у build.sh

- У build.sh додано окрему функцію для генерування документації:

```

generate_doc() {
    echo "Generating documentation..."
    mkdir -p docs
    cd docs || exit
    doxygen ../Doxyfile
    cd - > /dev/null
}

```

- Додано підтримку опції doc:

```

case "$1" in
    ...
    doc)
        generate_doc
        ;;
esac

```

6. Перевірка результату

- Після виконання скрипта build.sh із аргументом doc документація генерується в директорії docs у корені проекту.
- У CI/CD також додається команда для створення документації під час складання:

```

- name: Generate documentation
  run: cmake --build . --target doc

```

Висновок:

Під час додавання підтримки Doxygen було встановлено необхідний інструмент та створено файл конфігурації *Doxygen.in*. У коді додані коментарі до функцій, структур, класів і перерахувань відповідно до синтаксису Doxygen. Підтримка генерації документації інтегрована у файли *CMakeLists.txt* і скрипт *build.sh*, дозволяючи легко створювати документацію як локально, так і в CI/CD. У CI/CD процес автоматизовано через модифікацію YAML-файла GitHub Actions. Результатом є актуальна та доступна документація в директорії *docs* локально та під час складання проекту. Після ручного тестування розробленого функціоналу, додав зміни, закомітив їх, створив пул реквест та добавив викладача в рев'юери.